

Offline Multi-Model Chat Application - Documentation

Offline Multi-Model Chat Application

A local, privacy-first chatbot built with **Streamlit** and **LangChain** that runs entirely offline using local GGUF language models. It supports plain text conversation, **PDF-grounded Q&A (RAG)**, **audio transcription** (file upload or live mic recording), and includes an experimental **image understanding** module (LLaVA). It also ships with a live CPU/RAM/GPU hardware monitor, persistent multi-session chat history, and chat export.

Table of Contents

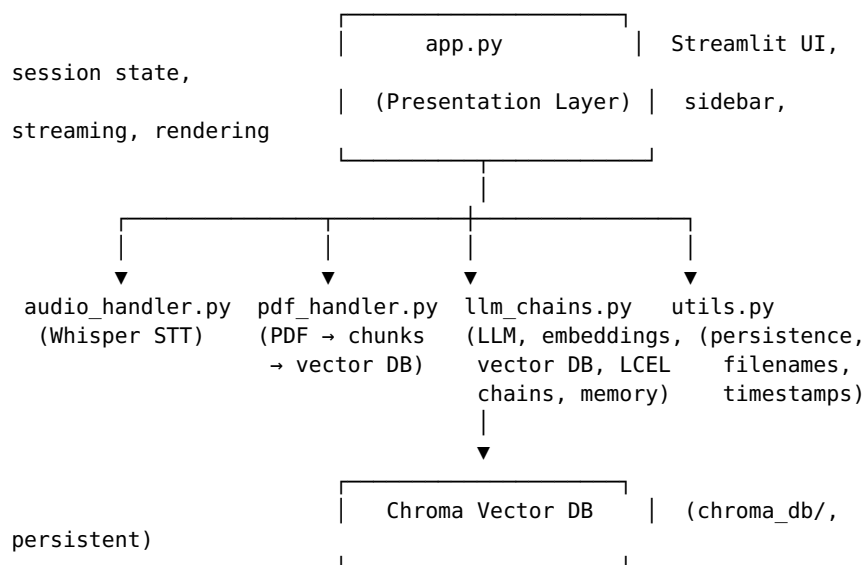
1. [Features](#)
 2. [Architecture Overview](#)
 3. [Project Structure](#)
 4. [Requirements](#)
 5. [Installation](#)
 6. [Configuration \(config.yaml\)](#)
 7. [Running the App](#)
 8. [Module Reference](#)
 9. [How the Pieces Fit Together](#)
 10. [Known Limitations](#)
 11. [Troubleshooting](#)
-

Features

Feature	Description
 Text Chat	Streaming conversation with a locally-run GGUF model via ctransformers.
 PDF Chat (RAG)	Upload PDFs, extract & chunk their text, embed them, and answer questions grounded in that content using a Chroma vector store.
 Audio Input	Transcribe uploaded audio files or live microphone recordings using OpenAI Whisper (transformers pipeline), then summarize or chat about the transcript.
 Multi-Model Support	Switch between configured models and available local .gguf files from the sidebar.
 Session Management	Auto-named chat sessions, persisted as JSON, with rename/delete support and a

🔗 Export	session picker. Download the current conversation as .txt or .json.
🖥️ Hardware Monitor	Live CPU, RAM, and (if available) GPU utilization/VRAM widget in the sidebar.
📄 Custom System Prompt	Editable per-session system prompt, with the option to persist it back into config.yaml.
📏 Adjustable Memory Window	Slider controlling how many recent turns are fed back into the model as context.

Architecture Overview



- **app.py** owns all UI state and orchestrates calls into the other modules. It never talks to the LLM or the vector store directly — it always goes through `llm_chains.py`.
- **llm_chains.py** is the core LLM/RAG engine: it builds the local LLM, the embeddings model, the Chroma vector store, and two LCEL (prompt | llm | parser) chains — one for plain chat, one for PDF-grounded chat.
- **pdf_handler.py** turns uploaded PDFs into embedded, searchable chunks inside the same vector store that `llm_chains.py` queries.
- **audio_handler.py** is a self-contained Whisper wrapper used for both file-upload and live-mic transcription.
- **utils.py** handles cross-cutting concerns: safe filenames, atomic JSON save/load of chat history, and timestamps.

Project Structure

```

project/
├── app.py                # Streamlit application entry point (UI
+ orchestration)
├── llm_chains.py         # LLM, embeddings, vector DB, and LCEL
chain definitions
├── pdf_handler.py        # PDF text extraction, chunking, and
ingestion into the vector DB
└── audio_handler.py      # Audio → text transcription (Whisper)
  
```

```

├─ prompt_templates.py      # Raw prompt templates used by the chat
chain
├─ utils.py                 # Chat history persistence, filename
sanitization, timestamps
├─ test.py                  # Minimal manual smoke test for the
vector store
├─ requirements.txt         # Python dependencies
├─ config.yaml              # App configuration (NOT included –
you must create it, see below)
├─ models/                  # Local GGUF model files (created
automatically)
├─ chat_history/            # Saved chat sessions as JSON (created
automatically)
└─ chroma_db/               # Persistent Chroma vector database
(created automatically)

```

Note: config.yaml is read by both app.py and llm_chains.py but is not part of the uploaded files. You must create it yourself — see [Configuration](#) below.

Requirements

From requirements.txt:

```

langchain==0.0.354
langchain-community==0.0.20
chromadb==0.4.22
ctransformers==0.2.27
InstructorEmbedding==1.0.1
sentence-transformers==2.2.2
torch==2.1.2
transformers==4.35.2
huggingface-hub==0.20.3
pypdfium2==4.24.0
librosa==0.10.1
PyYAML==6.0.1
streamlit==1.29.0
streamlit-mic-recorder==0.0.4

```

Additional dependencies required but not listed in requirements.txt (needed by the code as written):

Package	Used by	Purpose
psutil	app.py	CPU/RAM monitoring
pynvml	app.py	GPU monitoring (optional — app degrades gracefully without it)
langchain-core, langchain-text- splitters	llm_chains.py, pdf_handler.py	Newer LangChain APIs (with fallback imports to the old paths)

Install these alongside the pinned requirements:

```
pip install psutil pynvml langchain-core langchain-text-splitters
```

⚠ **Version note:** langchain==0.0.354 is quite old. The code in llm_chains.py, pdf_handler.py, and utils.py uses try/except ImportError blocks to support both the legacy (langchain.schema...) and modern (langchain_core...) import paths, so it should work with a newer LangChain install too —

but the pinned version in requirements.txt predates several of the imports used (langchain_core, langchain_text_splitters). Consider upgrading langchain/langchain-community and adding langchain-core + langchain-text-splitters explicitly for a smoother install.

Installation

```
# 1. Clone/copy the project files into a folder
cd project/

# 2. Create a virtual environment
python -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt
pip install psutil pynvml langchain-core langchain-text-splitters

# 4. Create config.yaml (see next section)

# 5. Place your GGUF model(s) inside models/
mkdir -p models
# copy your .gguf file(s) here

# 6. Run the app
streamlit run app.py
```

Configuration (config.yaml)

Both app.py and llm_chains.py load a config.yaml file from the project root at import time. Create one like this:

```
# Path to the default large model used by CTransformers
model_path:
  large: "models/your-model.Q4_K_M.gguf"

# Model family recognized by ctransformers (e.g. llama, mistral,
mpt...)
model_type: "mistral"

# Passed straight into CTransformers(config=...)
model_config:
  max_new_tokens: 512
  temperature: 0.7
  context_length: 2048
  gpu_layers: 0      # increase if you have CUDA support

# Embeddings model used for the PDF vector store
embeddings_path: "hkunlp/instructor-large"

# Directory scanned for switchable .gguf files (sidebar dropdown)
models_dir: "models"

# Directory where chat session JSON files are saved
chat_history_path: "chat_history"

# Default list shown in the "Active Model" selector
available_models:
  - "default"

# Default system prompt shown/edited in the sidebar
```

```

        system_prompt: "You are a helpful, honest, and concise AI
assistant."

        # Default number of MESSAGES (not turns) kept in the memory window
slider
        memory_window: 10

        # Number of conversation TURNS (human+AI pairs) used to build
{history}
        # inside the prompt template (llm_chains.py)
        memory_window_turns: 3

```

Important distinction: `memory_window` (in `app.py`) counts **messages**, while `memory_window_turns / DEFAULT_MEMORY_K` (in `llm_chains.py`) counts **turns** (a human+AI pair). `app.py`'s `load_chain()` converts the slider value from messages to turns (`window_messages // 2`) before passing it to the chain — you don't need to reconcile these manually, just be aware the two numbers mean different things.

Running the App

```
streamlit run app.py
```

This opens the app in your browser (default `http://localhost:8501`). On first load:

- `models/` and `chat_history/` are created automatically if missing.
 - The **first message** you send creates a new named session (the LLM auto-generates a short 3–5 word title from your message), saved to `chat_history/<title>.json`.
 - Toggling  **PDF Chat Mode** in the sidebar switches from the plain chat chain to the retrieval-augmented (PDF) chain.
-

Module Reference

`app.py` — UI & Orchestration

The Streamlit entry point. Responsibilities:

- **Session state initialization** — a defaults dict seeds all keys used across reruns (history, selected model, memory window, stop event, etc.).
- **Sidebar** — model selector, GGUF file picker, system prompt editor, memory window slider, PDF Chat toggle, session list (select/rename/delete), export buttons, audio/PDF file uploaders, and the live hardware monitor.
- **`stream_response()`** — drives token-by-token streaming from the active chain into a chat bubble, with a typing-cursor animation, a stop button (via a `threading.Event`), and graceful fallback to a blocking `.run()` call if streaming isn't supported by the underlying chain.
- **`render_message()`** — renders a saved/streamed message, splitting on triple backticks to render fenced code blocks with `st.code()`, and adds a "📄 Copy" button for assistant replies.
- **`load_chain()`** — builds a fresh `ChatChain` or `PDFChatChain` (from `llm_chains.py`) every rerun, converting the sidebar's memory-window slider from messages to turns.
- **Audio handlers** — both uploaded audio files and live mic recordings are transcribed via `audio_handler.transcribe_audio()`; the resulting transcript is summarized by the LLM using a clearly-delimited prompt (to avoid the model echoing the transcript

instead of summarizing it).

- **PDF ingestion** — uploaded PDFs are sent to `pdf_handler.add_documents_to_db()`; success/failure status is surfaced in the sidebar, distinguishing “no extractable text” (e.g. a scanned PDF) from a hard failure.
- **Session persistence** — `save_chat_history()` writes the **full** (non-windowed) history to disk after every exchange, guarding against Streamlit’s rule that widget-owned session-state keys (like the session selectbox) can’t be reassigned directly from code.

`llm_chains.py` — LLM, Embeddings & RAG Chains

- **`create_llm()`** — builds the local CTransformers LLM from `config.yaml`, cached via `st.cache_resource` so the (slow, memory-heavy) GGUF model is loaded only once per process, not on every Streamlit rerun.
- **`create_embeddings()`** — builds a HuggingFaceInstructEmbeddings model, also cached.
- **`load_vectordb()`** — creates/connects to the persistent Chroma client at `chroma_db/`, disables ChromaDB’s telemetry, and is cached to avoid the `ValueError: An instance of Chroma already exists...` singleton conflict that can occur if `PersistentClient` is created twice with different settings in the same process (a fallback except still handles it defensively if it ever slips through).
- **`_format_history_text()`** — renders the last *k* human/AI turns into a plain-text transcript (Human: ... / AI: ...), since the prompt templates expect `{history}` as a string, not a list of message objects.
- **`ChatChain`** — the plain-chat LCEL pipeline (`prompt | llm | StrOutputParser`). Exposes `.stream()` (real token streaming) and `.run()` (blocking fallback). Reads `chat_history.messages` **read-only** — it never mutates the shared StreamlitChatMessageHistory object, to avoid accidentally truncating the saved conversation.
- **`PDFChatChain`** — retrieval-augmented version: retrieves the top-k relevant chunks from the Chroma vector store for the user’s question, injects both that context and the recent conversation history into a Mistral-style prompt, then streams/runs the answer the same way as `ChatChain`.

`pdf_handler.py` — PDF Ingestion

- **`extract_text_from_pdf()`** — extracts all text from a PDF’s raw bytes using `pypdfium2`; raises a clear `ValueError` for corrupt/password-protected files instead of a cryptic low-level exception.
- **`get_pdf_texts()`** — extracts text from each uploaded file, returning `(filename, text)` tuples so problems can be traced back to a specific file.
- **`get_text_chunks()`** — splits text using `RecursiveCharacterTextSplitter` (1000 chars, 150 overlap).
- **`get_document_chunks()`** — wraps each chunk in a `LangChain Document`, tagging it with `metadata={"source": filename}` for traceability.
- **`add_documents_to_db()`** — the main ingestion entry point. Extracts → chunks → embeds → stores. Explicitly detects and reports the “PDF has no extractable text” case (e.g. a scanned image-only PDF with no OCR layer) as a clear warning rather than silently storing nothing or crashing deep inside ChromaDB.

`audio_handler.py` — Speech-to-Text

- **`convert_bytes_to_array()`** — loads raw audio bytes into a 16kHz mono waveform via `librosa`.
- **`transcribe_audio()`** — runs OpenAI Whisper (`openai/whisper-small`)

through the `transformers pipeline()` API, automatically using CUDA if available, and returns the transcribed text.

△ The Whisper pipeline is rebuilt on every call to `transcribe_audio()` rather than being cached — this reloads the model from disk/HF cache on every transcription. Consider caching this similarly to `create_llm()/create_embeddings()` if transcription is used frequently.

`prompt_templates.py` — Prompt Templates

- **memory_prompt_template** — the Mistral-style instruction template (`<s>[INST] ... [/INST]`) used by `ChatChain`. Expects `{history}` and `{human_input}` as plain strings.

`utils.py` — Persistence Utilities

- **sanitize_filename()** — strips characters forbidden in Windows filenames, collapses whitespace, and caps length, falling back to a timestamp if the result is empty.
- **save_chat_history_json()** — writes chat history to disk **atomically**: writes to a temp file in the same directory, `fsyncs` it, then uses `os.replace()` to atomically swap it into place — so a crash mid-write can never leave a corrupted/truncated history file. Also preserves any non-human/AI message type instead of silently dropping it.
- **load_chat_history_json()** — loads history from disk, returning `[]` on any error (missing file, empty file, invalid JSON) rather than raising.
- **get_timestamp()** — returns a `YYYY-MM-DD_HH-MM-SS` string, used as a fallback session/file name.

`test.py` — Manual Smoke Test

A minimal script that loads the vector DB and runs a similarity search for "Hover dataset" — useful as a quick sanity check that embeddings + Chroma are working, independent of the Streamlit UI.

How the Pieces Fit Together

Text chat flow: `app.py` (user types) → `load_chain()` → `llm_chains.ChatChain.stream()` → `create_llm()` (cached `CTransformers`) → tokens streamed back → `stream_response()` renders them live → `save_chat_history_json()` persists the full history.

PDF chat flow: Upload → `pdf_handler.add_documents_to_db()` → `pdf_handler.extract_text_from_pdf()` → `get_text_chunks()` → `get_document_chunks()` → `llm_chains.load_vectordb()` → embedded & stored in Chroma. Then, on each question: `llm_chains.PDFChatChain._retrieve_context()` pulls the top-k matching chunks → both context and conversation history are injected into the PDF prompt template → answer streamed back.

Audio flow: Upload or record → `audio_handler.transcribe_audio()` (Whisper) → transcript is either summarized (file upload) or sent directly as a chat message (mic recording) → same streaming/history path as text chat.

Known Limitations

- **Whisper pipeline is reloaded on every transcription call** (no caching), unlike the LLM/embeddings which are cached.
- **requirements.txt is missing some runtime dependencies** actually imported by the code (psutil, pynvml, langchain-core, langchain-text-splitters) — install these manually (see [Requirements](#)).
- **config.yaml is not included** in the project files and must be created manually before the app will start (both app.py and llm_chains.py fail fast with a clear error if required keys are missing).
- **GPU monitoring requires an NVIDIA GPU + pynvml**; on unsupported hardware the app gracefully falls back to showing “Not Available” instead of crashing.

Troubleshooting

Symptom	Likely Cause	Fix
RuntimeError: config.yaml is missing required key ...	config.yaml doesn't have model_path.large, model_type, or model_config	Add the missing key(s) — see Configuration
ValueError: An instance of Chroma already exists for ... with different settings	A PersistentClient for chroma_db/ was created earlier in the same process with different settings (usually only if caching was bypassed)	Restart the Streamlit server fully (not just a hot- reload/code save) You'll now see an explicit warning naming the affected file(s); OCR the PDF first, or use a text-based PDF
PDF upload succeeds but the model says “I don't have access to the document”	The PDF has no extractable text layer (e.g. a scanned image with no OCR)	Copy your .gguf model file(s) into that folder
“No GGUF models found” in the sidebar	No .gguf files inside models_dir (default models/)	Already addressed — reruns now only happen when a brand- new session is created
Duplicate question/answer appears in chat	(Fixed in this version) previously caused by an unconditional st.rerun() after a normal text reply	This is an intentional fallback to .run() (blocking) — check that the underlying CTransformers LLM/langchain version supports streaming if you want live
Streaming doesn't show tokens live, jumps straight to full answer	The chain's .stream() raised AttributeError/NotImplementedError	

GPU monitor shows "Not Available" despite having a GPU		pynvml not installed, or pynvml.nvmlInit() failed	tokens pip install pynvml; ensure NVIDIA drivers are installed and accessible